

Postgres pgx Integration Suite — REAL RUN evidence (§11.4.83)

Run date: 2026-06-23 **Run host:** nezha.local (Linux x86_64, go1.26.2, rootless podman 5.7.1 + podman-compose) **Driver host:** mistborn (macOS) — has go1.26.2 but NO container runtime; tests were rsynched + run on nezha where go1.26 AND rootless podman both exist.

Why this matters

The server's production persistence target is the pgx/PostgreSQL Repository (server/internal/store/postgres.go) and the rollout PostgresStore (server/internal/rollout/postgres.go). These are covered ONLY by // go:build integration tests, which the default go test ./... run does NOT build/execute — so the headline coverage (store 47.9% / rollout 28.2%) understated the real figure because the pgx code paths were never run. This run executes them.

Boot mechanism (cited)

The integration tests boot Postgres ON-DEMAND via the digital.vasic.containers submodule — never a manual podman/compose step (§11.4.76 on-demand-infra invariant):

- server/internal/store/postgres_integration_test.go — imports digital.vasic.containers/pkg/{boot,compose,endpoint,health,runtime,logging}; runtime.AutoDetect(ctx) finds podman; compose.NewDefaultOrchestrator + boot.NewBootManager(...).BootAll(ctx) brings up the service defined in server/deploy/postgres.compose.yml (image postgres:16-alpine, host port **55432**, DSN postgres://helix:helix@localhost:55432/helix_ota?sslmode=disable).
- server/internal/store/postgres_fabric_integration_test.go — fabric registry (leases/evidence) against the same DB.
- server/internal/rollout/postgres_integration_test.go — boots server/deploy/postgres-rollout.compose.yml (host port **55445**).
- server/internal/{store,rollout}/pg_itest_lock_test.go — a fixed-path flock serializes the shared Postgres lifecycle across the two integration BINARIES (§11.4.119 single resource owner) so parallel go test packages don't race compose up/down.

go.mod local deps (replace directives): digital.vasic.containers => ../containers, digital.vasic.http3 => ../submodules/http3, ota-protocol => ../submodules/ota-protocol. The ota-artifact-

validator, ota-rollout-engine, ota-telemetry-schema bricks are published modules fetched via GOPROXY. All four local dirs were rsynced to nezha preserving the relative layout.

Run path that worked

Path (b): rsync server/ + containers/ + submodules/http3/ + submodules/ota-protocol/ to nezha → go mod download → go test -tags integration. (Path (a) DOCKER_HOST=ssh:// was NOT viable — the test connects pgxpool to localhost:55432, not to the runtime host, so a remote runtime would not be reachable from the Mac's localhost.)

REAL result — PASS, with measured coverage

```
ok internal/store      25.937s coverage: 85.5% of statements in ./
internal/store/...
ok internal/rollout    7.953s coverage: 83.1% of statements in ./
internal/rollout/...
```

Combined run (int_test.log): all integration + unit tests PASS, EXIT=0.
TestPostgresRepositoryContract_Integration PASS (6.99s) — container runtime: podman, boot summary: started=1 discovered=0 failed=0.
TestPostgresFabricRegistry_Integration PASS (13.39s) — exercised real DB UNIQUE partial index uq_fabric_lease_active (lease exclusivity) + fabric_evidence CHECK constraint (0-byte INSERT rejected SQLSTATE 23514). TestPostgresStoreScenario_Integration PASS (6.59s) — boot summary: started=1 failed=0.

Package	Default go test (no integration tag)	Integration run (pgx paths executed)
internal/store	47.9%	85.5%
internal/rollout	28.2%	83.1%

pgx production paths genuinely exercised (func coverage)

store/postgres.go: CreateRelease 100%, GetRelease 100%, GetDeployment 100%, CreateDeployment 100%, ActiveDeploymentForTarget 100%, AppendTelemetry 100%, GetDevice 100%, GetDeviceByHardwareID 100%, CreateDevice 87.5%, ListReleases 86.4%, UpdateDeployment 85.7%, ... (full breakdown in coverage_func_full.txt). rollout/postgres.go: Save 73.3%, Load 79.2%, NewPostgresStore 85.7%. Residual uncovered: ListDevices 0.0%, NewPostgresRepositoryFromPool 0.0%, NewPostgresStoreFromPool 0.0% — not driven by the contract scenario.

Evidence files

- `integration_test_run.log` — combined `-v` run (all tests + boot summaries, `EXIT=0`)
- `store_pkg_coverage.log` / `rollout_pkg_coverage.log` — per-package coverage lines
- `coverage_func_full.txt` — `go tool cover -func full` per-function breakdown (both pkgs)

Cleanup (§11.4.14)

postgres compose containers + lock files torn down on nezha after the run; staging dir `~/helix_ota_itest` left in place (not the project tree). See report for teardown confirmation.